

数据结构第三次作业

3.29

```
typedef struct{
    Elemtype *base;
    int front,rear;
    int tag;//初始化应置为0
}SqQueue;

Status EnQueue(SqQueue *q,Elemtype e){
    if(q->front == q->rear && q->tag){
        printf("Queue Full!");
        return ERROR;
    }

    q->base[q->rear++] = e;
    q->rear %= MAXQSIZE;
    q->tag = 1;
    return OK;
}

Status DeQueue(SqQueue *q,Elemtype *e){
    if(q->front == q->rear && !(q->tag)){
        printf("Queue Empty!");
        return ERROR;
    }

    *e = q->base[q->front++];
    q->front %= MAXQSIZE;
    q->tag = 0;
    return OK;
}
```

对比另一种方法(牺牲一个存储位置) 时间方面：相比于另一种方法，由于每次判断需要判断tag（front == rear 的时候），以及成功增删之后需要更新tag，如果是元素较多的情形，可能会导致时间开销更大的情况，但是实际上，时间复杂度均为 $O(1)$ ，时间方面的影响可以忽略不计 空间方面：tag标志方法，可以避免有一个空间无法存储元素的情况。如果是小元素，队列长度较大，对空间的节约可忽略不计；但是如果是大元素，队列较短，采取这种方法，可以避免有一个空间没有被利用，节约空间

3.31

```
//双端非循环队列
typedef struct{
    char *base;
    int front,rear;
}SqQueue;
```

```

Status IsPalindrome(char *str){
    SqQueue q;

    if(!(q.base = (char*)malloc(MAXQSIZE*sizeof(char)))){
        printf("Malloc Fail!");
        return ERROR;
    }
    q.front = q.rear = 0;

    //默认不会出现字符串中无@以及str = NULL的情形
    while(*str != '@')
        Enqueue(&q,*(str++));

    q.rear--;
    while(q.front < q.rear)
        if(q.base[q.front++] != q.base[q.rear--]){
            free(q.base);
            return FALSE;
        }

    free(q.base);
    return TRUE;
}

```

4.22

这题我接口好像不太对，我的是t传入一个块链，s传入一个字符串

```

#define BUFF_SIZE 4

typedef struct StrNode{
    char *buff;
    struct StrNode *next;
}StrNode,*StrList;

void CreateMyList(char *str,StrNode **head,StrNode **tail){
    *head = (StrNode*)malloc(sizeof(StrNode));
    (*head)->buff = NULL;
    (*head)->next = NULL;

    *tail = *head;

    int len = strlen(str);
    int idx = 0;

    while(idx < len){
        StrNode *node = (StrNode*)malloc(sizeof(StrNode));
        node->buff = (char*)malloc(BUFF_SIZE * sizeof(char));
        node->next = NULL;
    }
}

```

```
        for(int i = 0; i < BUFF_SIZE; i++){
            if(idx < len)
                node->buff[i] = str[idx++];
            else
                node->buff[i] = '#';
        }

        (*tail)->next = node;
        *tail = node;
    }
}

Status insertBuff(StrList sl,char c,char *str){
    StrNode *p = sl->next,*pre = sl;
    int i;
    while(p){
        for(i = 0; i < BUFF_SIZE; i++){
            if(p->buff[i] == c)
                break;

            if(i == BUFF_SIZE){
                pre = p;
                p = p->next;
            }
            else
                break;
        }

        StrNode *head,*tail;

        CreateMyList(str,&head,&tail);

        if(!p)
            pre->next = head->next;
        else{
            StrNode *p1 = (StrNode*)malloc(sizeof(StrNode)),
                *p2 = (StrNode*)malloc(sizeof(StrNode));
            p1->buff = (char*)malloc(BUFF_SIZE*sizeof(char));
            p2->buff = (char*)malloc(BUFF_SIZE*sizeof(char));
            for(int m = 0; m <= i; m++){
                p1->buff[m] = p->buff[m];
                p2->buff[m] = '#';
            }
            for(int m = i+1; m < BUFF_SIZE; m++){
                p2->buff[m] = p->buff[m];
                p1->buff[m] = '#';
            }

            p1->next = head->next;
            pre->next = p1;
            tail->next = p2;
            p2->next = p->next;
        }
    }
}
```

```
    free(p);  
    return OK;  
}
```

4.31

```
#define min(a,b) ((a)<(b)?(a):(b))  
#define max(a,b) ((a)>(b)?(a):(b))  
  
char **MaxSubstr(char *s,char *t,int *size){  
    char **res;  
    int max_len = 0,nums = 0,  
        slen = strlen(s),tlen = strlen(t),  
        dp[slen+1],idxarr[20];  
    //idxarr用于存储所有符合要求的子串的起始下标,nums是子串个数  
    memset(dp,0,sizeof(dp));  
  
    for(int idx = 1-tlen; idx < slen; idx++){  
        int uplim = min((slen-idx),tlen),sublim = max(-  
idx,0),subdp[tlen+1];  
        memset(subdp,0,sizeof(subdp));  
  
        for(int i = sublim; i < uplim; i++)  
            if(s[idx+i] == t[i])  
                subdp[i+1] = subdp[i] + 1;  
  
        for(int i = sublim; i < uplim; i++)  
            dp[idx+i+1] = max(dp[idx+i+1],subdp[i+1]);  
    }  
  
    for(int i = 0; i < slen; i++){  
        if(dp[i+1] > max_len){  
            nums = 0;  
            max_len = dp[i+1];  
        }  
  
        if(max_len == dp[i+1])  
            idxarr[nums++] = i - max_len + 1;  
    }  
  
    if(!max_len){  
        *size = 0;  
        return NULL;  
    }  
  
    res = (char**)malloc(nums*sizeof(char*));  
    for(int i = 0; i < nums; i++){  
        int idx = idxarr[i];  
        res[i] = (char*)malloc((max_len+1)*sizeof(char));  
    }  
}
```

```
        for(int j = 0; j < max_len; j++)
            res[i][j] = s[idx + j];

        res[i][max_len] = 0;
    }

    *size = nums;
    return res;
}
```